14: Memory Management

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Some material taken from EECS370 at U. of Michigan

Announcements

- P4 is out!
- Exam Tuesday!

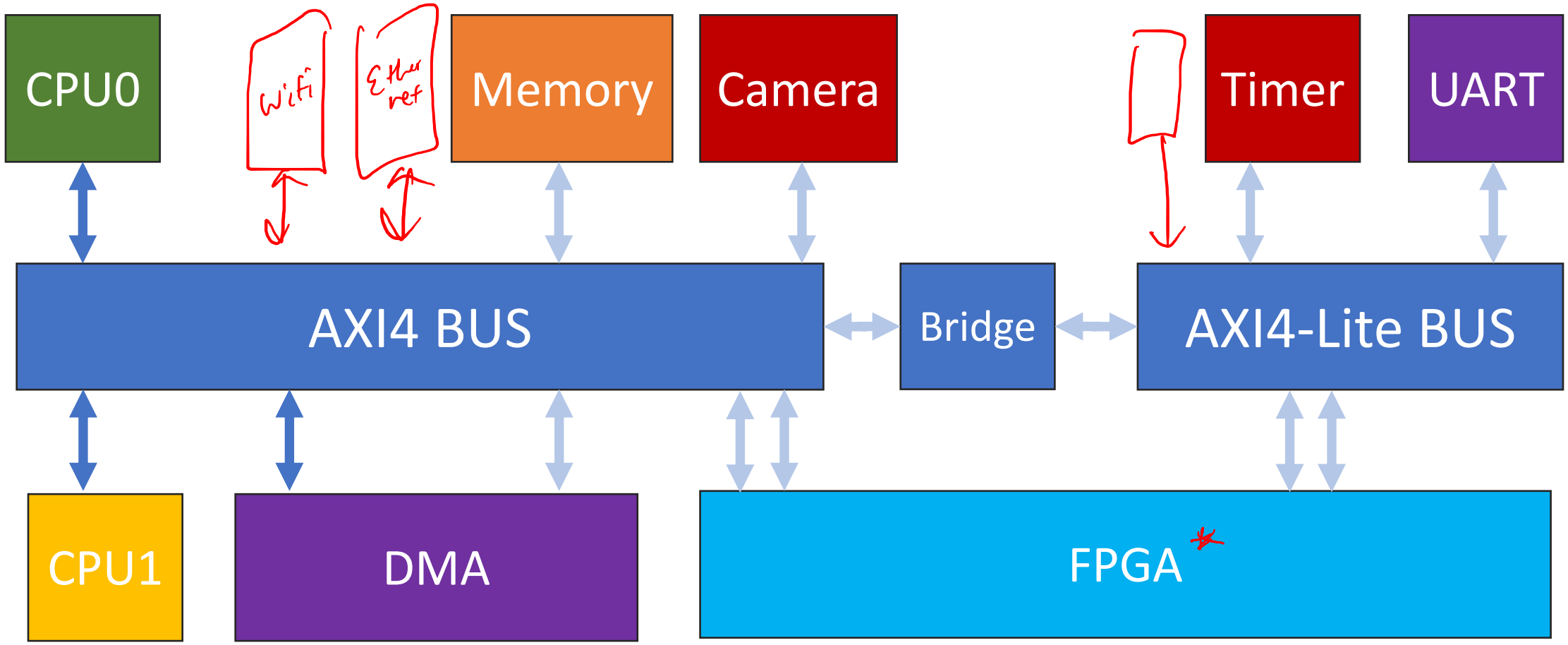
P4 Overview

Popcount in C example

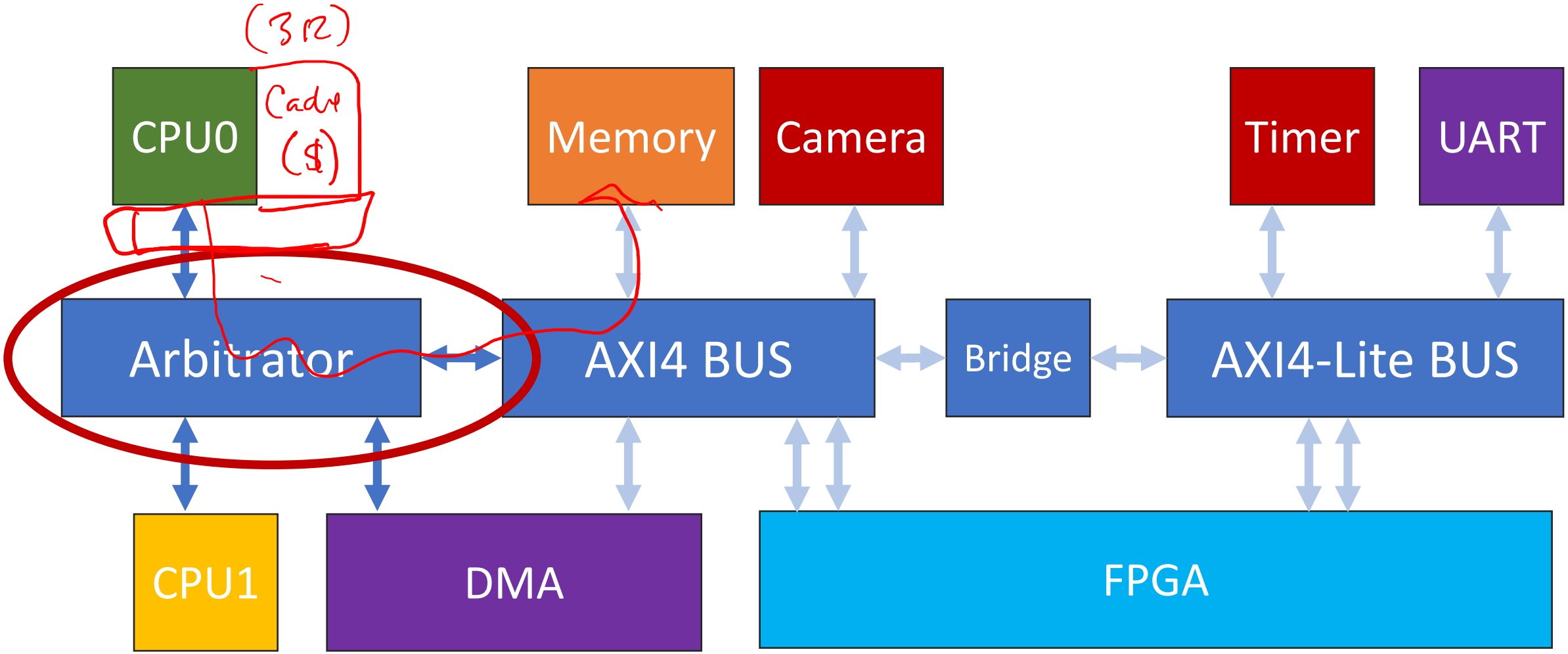
Machine Model, V0

files?

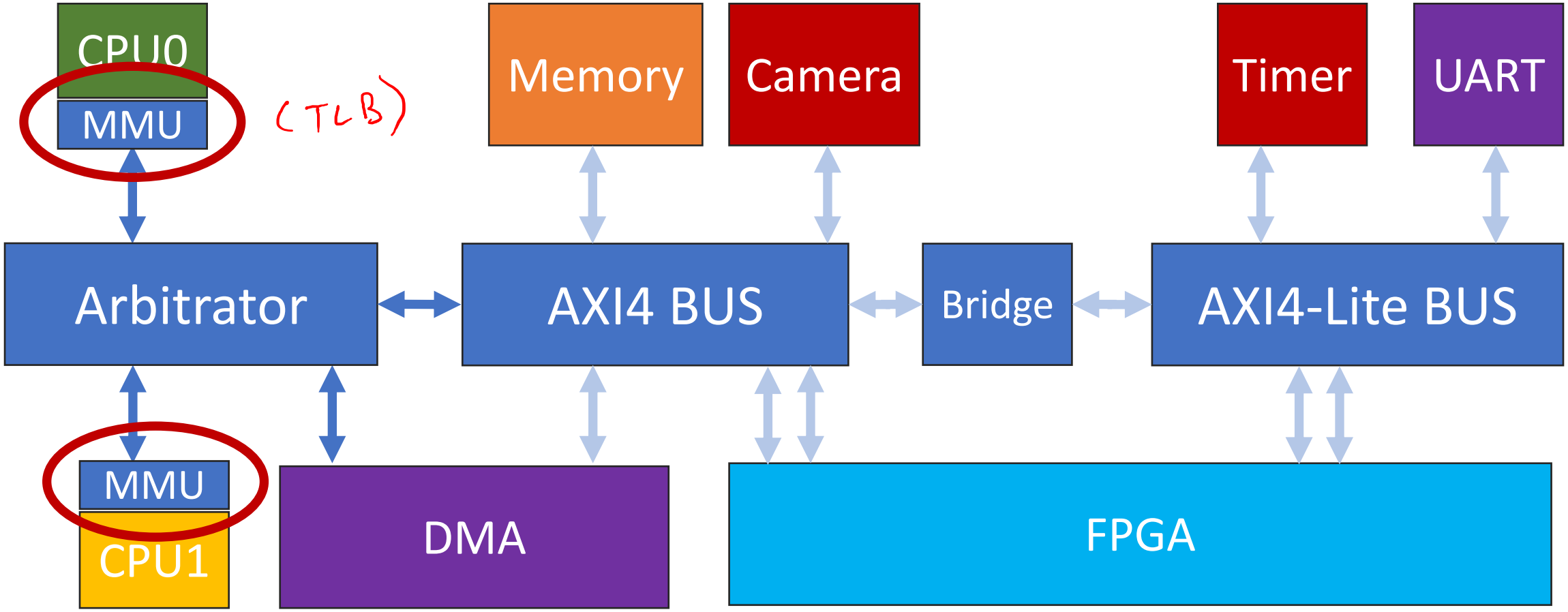
Machine Model, V1



Machine Model, V2

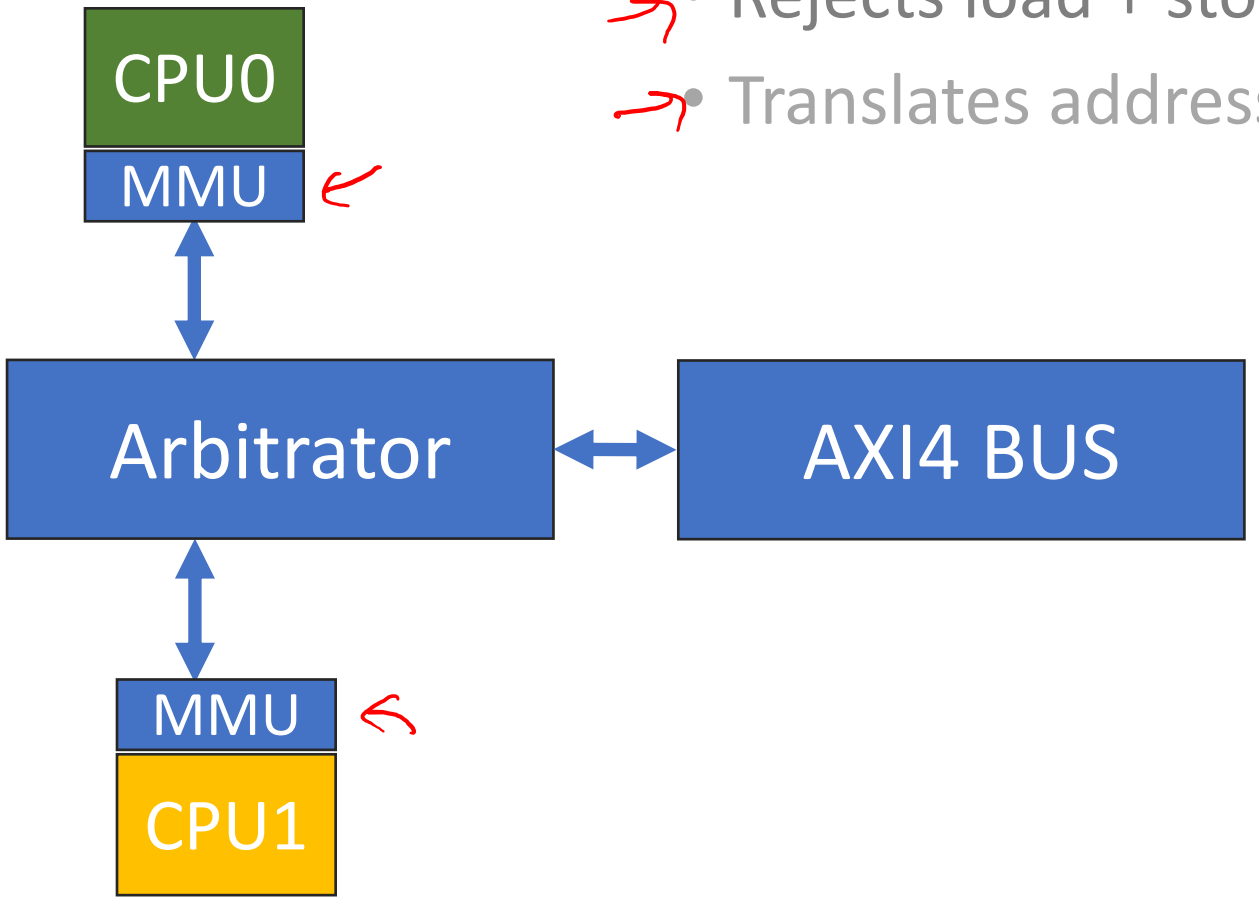


Machine Model, V3

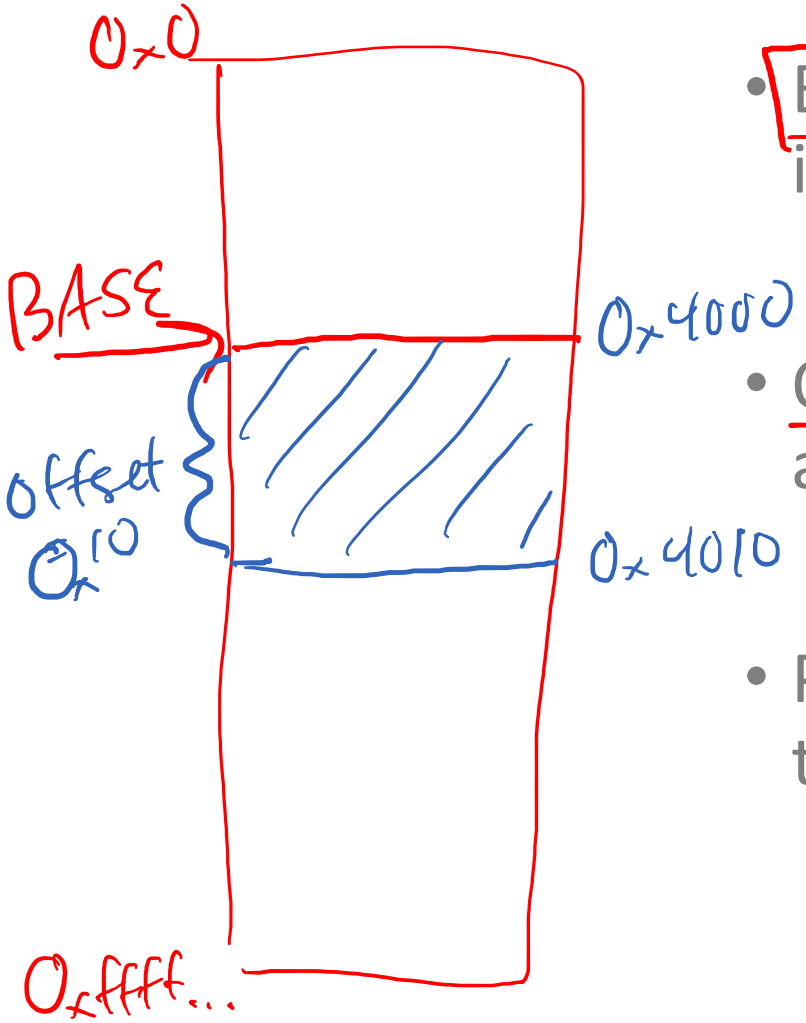


MMU: Memory Management Unit (TLB)

- Rejects load + stores that are "unauthorized"
- Translates addresses (Later)



MMUs track the following things



0x4000_0000

- BASE ADDRESS: the start of a memory region that is allowed through the MMU

2 Byte region → 0x4000-0000 – 0x4000-0002
=> offset = 0x2

- OFFSET: the size of a memory region that is allowed through the MMU

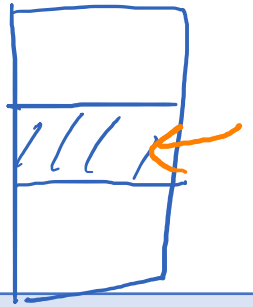
4KB → 0x4000-0000 – 0x4000_0100

- Permission: the type of access that is allowed through the MMU

Basic MMU Table

Base = 4000

Base + offset 4010



* Other permissions

Execute

CPU0

BASE Addr	Offset	Permissions
0x4000	0x10	Read / Write
0x6000	0x20	Read / ~~Write~~
"code"		Read / ~~Write~~
"constants"	"RO data"	Read / ~~Write~~

BUS

load 0x4000 ✓

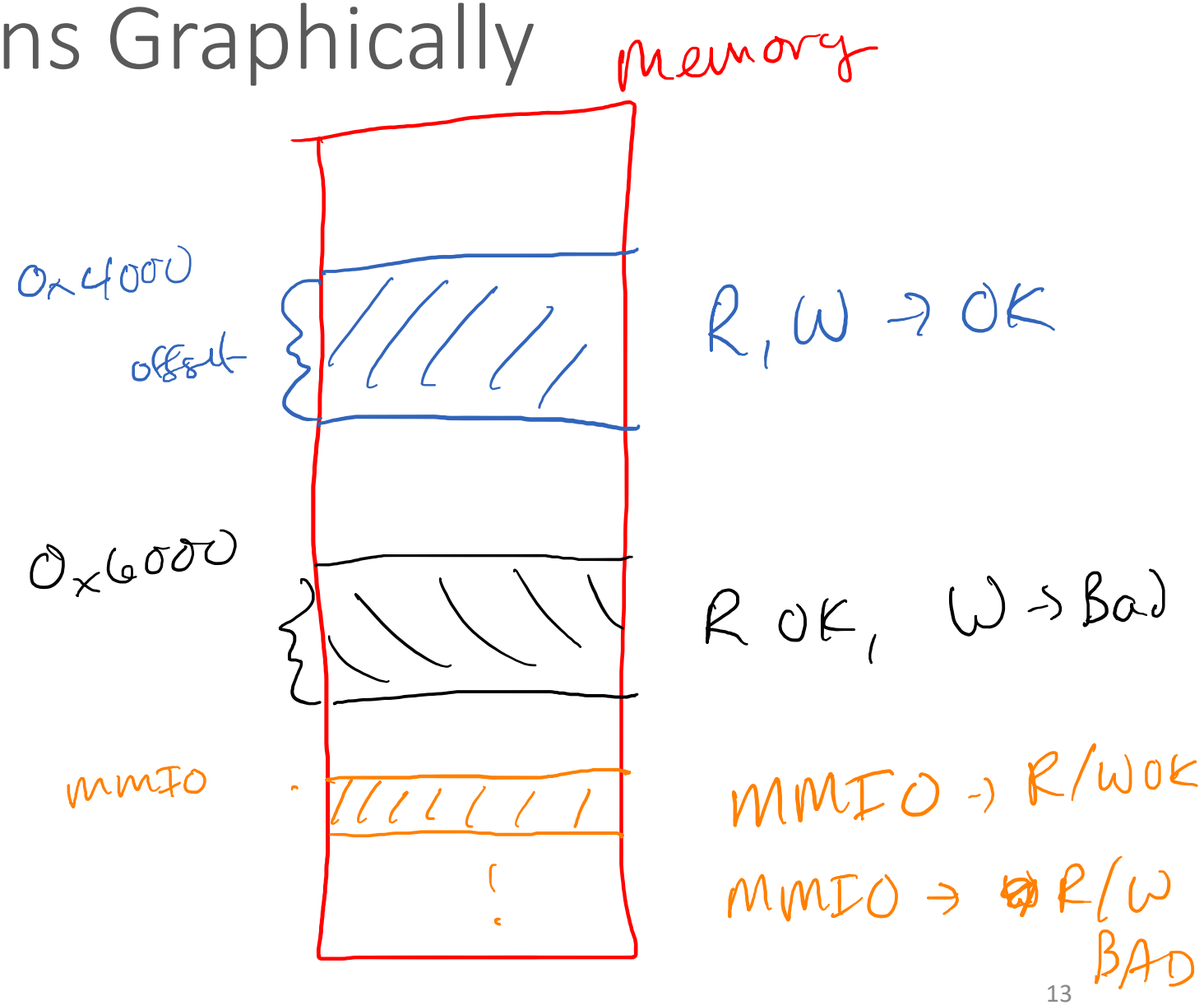
store 0x4008 ✓

load 0x5000 *

load 0x6000 ✓

store 0x6000 *

Memory Protections Graphically



Why do this?

- Security
  - Keep you from modifying the code
  - Keep you from executing the data
- Concurrency (?)

Micro-controllers
MPU
memory Perfection Unit

Separating multiple applications?

- A) What if two applications want to use the same memory address?
- B) How do I prevent your application from modifying my memory?

Two application test..

```c
#include <stdio.h>
#include <stdlib.h>

volatile int avalue = 2;

int main ()
{

    while (avalue == 2) { ; }
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>

volatile int avalue = 3;

int main ()
{

    while (avalue == 3) { ; }
    return 0;
}
```

Two application test..

```
gcc -g -O0 test_1.c -o test_1.out
objdump -DSs test_1.out > test_1.dis
gcc -g -O0 test_2.c -o test_2.out
objdump -DSs test_2.out > test_2.dis

vi test_1.dis test_2.dis
```

Two application test..

```
00011008 <avalue>:
volatile int avalue = 2;
   11008:       00000002        andeq   r0, r0, r2
```

```
00011008 <avalue>:
volatile int avalue = 3;
   11008:       00000003        andeq   r0, r0, r3
```

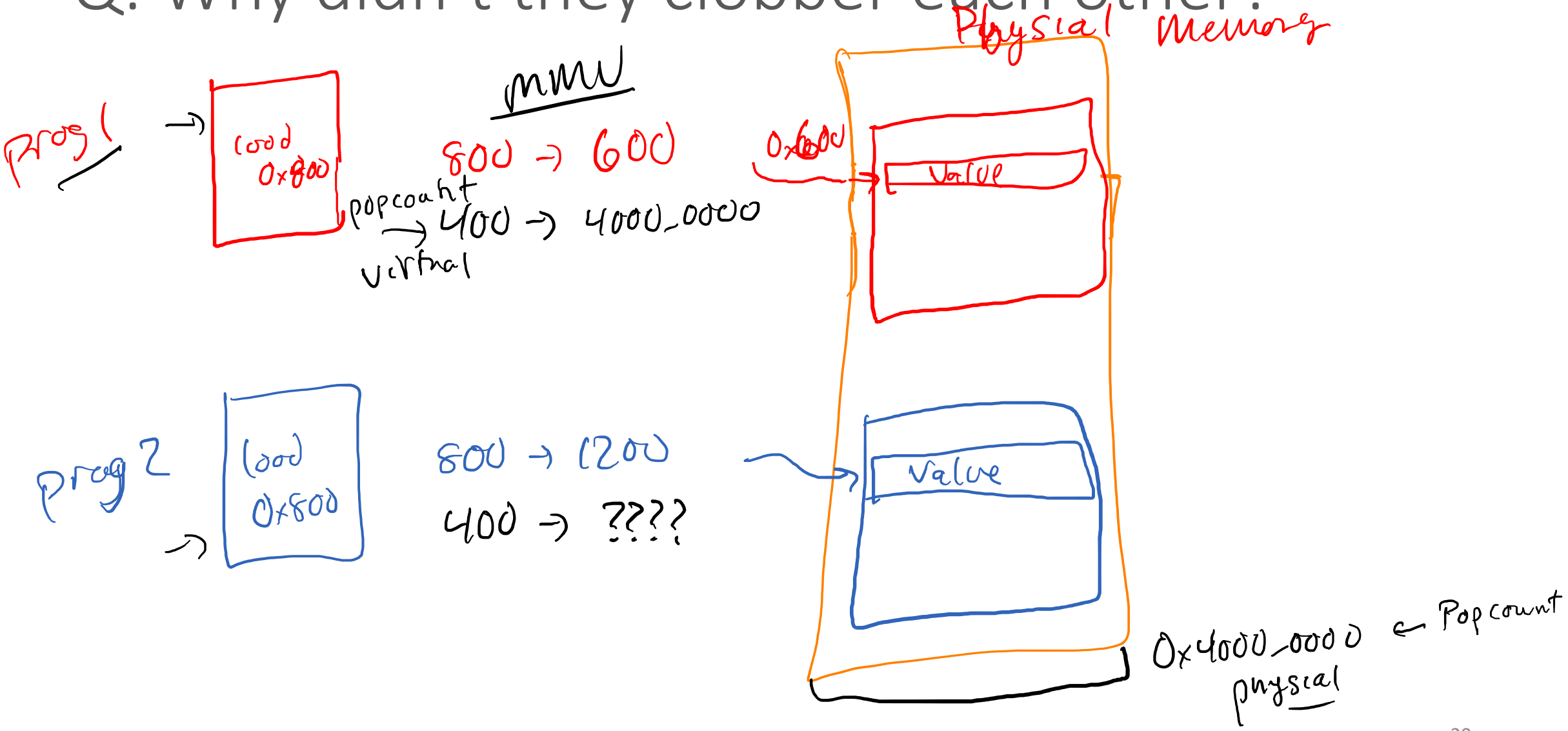
Two application test..

```
./test_1.out &
./test_2.out &

top
```

Swany: Mentioned 201

Q: Why didn't they clobber each other?



Q: Why didn’t they clobber each other?

- A: MMUs are doing something else… “Virtual Memory”

Virtual memory with an MMU

- MMU automatically translates each memory reference from a

virtual address

(which the programmer sees as a huge array of bytes)

to a

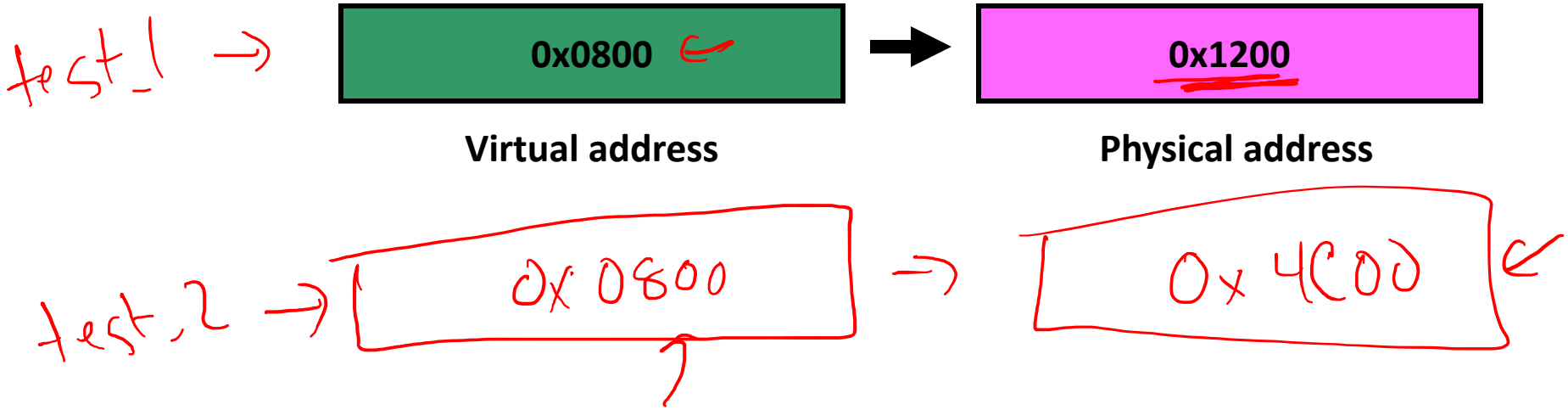
physical address

(which the hardware uses to identify where the storage actually resides)

Basics of Virtual Memory

- Any time you see the word virtual in computer science and architecture it means "using a level of indirection"

- Virtual memory hardware changes the virtual address the programmer sees into the physical one the memory chips see



Virtual Memory View

- Virtual memory lets the programmer address a memory array larger than the DRAM available on a particular computer system OS (pages)

- Virtual memory enables multiple programs to share the physical memory without:
  - Knowing other programs exist (**transparency**)
  - Worrying about one program modifying the data contents of another (**protection**)

Managing virtual memory

OS

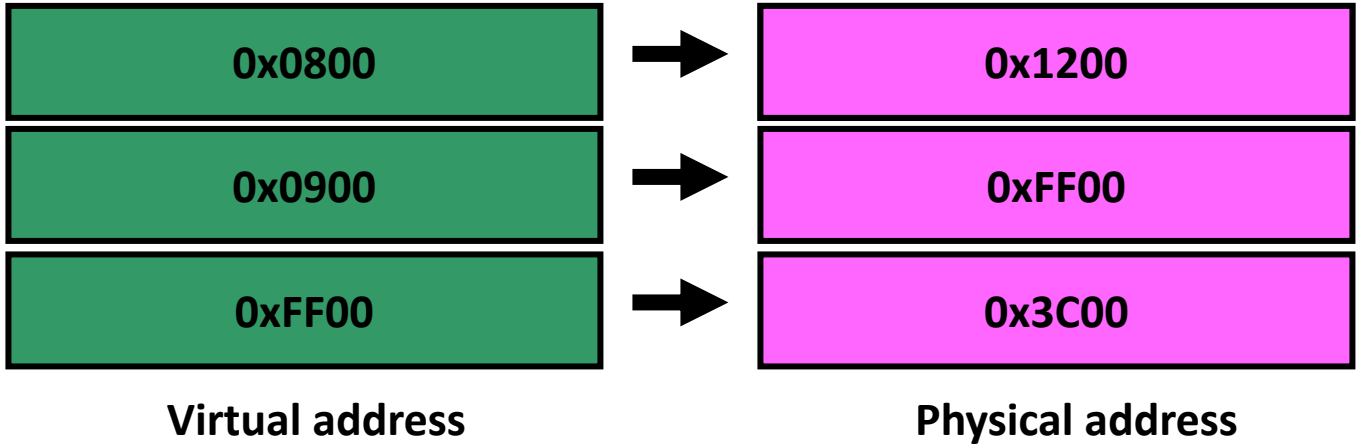
- Managed by hardware logic ***and*** operating system software
  - Hardware for speed
  - Software for flexibility and because disk storage is controlled by the operating system

- The hardware must be designed to support VM

Pyng

Translation Look-aside Buffers

- Buffer common Virtual->Physical translations in a **Translation Look-aside Buffer (TLB)**, a fast cache memory dedicated to storing a small subset of valid translations
- 16-512 entries common
- Generally has low miss rate (< 1%)



MMU Address Translation

6000
7000

unity mapping

Virtual Address
(32-bit)

Virtual Base	Offset

0x69ff | 0004
2000 | 0008

v	Virtual Base	Physical Base
	69ff	4000
	3f00	4f00
	2000	2000
	6000	8000
	7000	8000

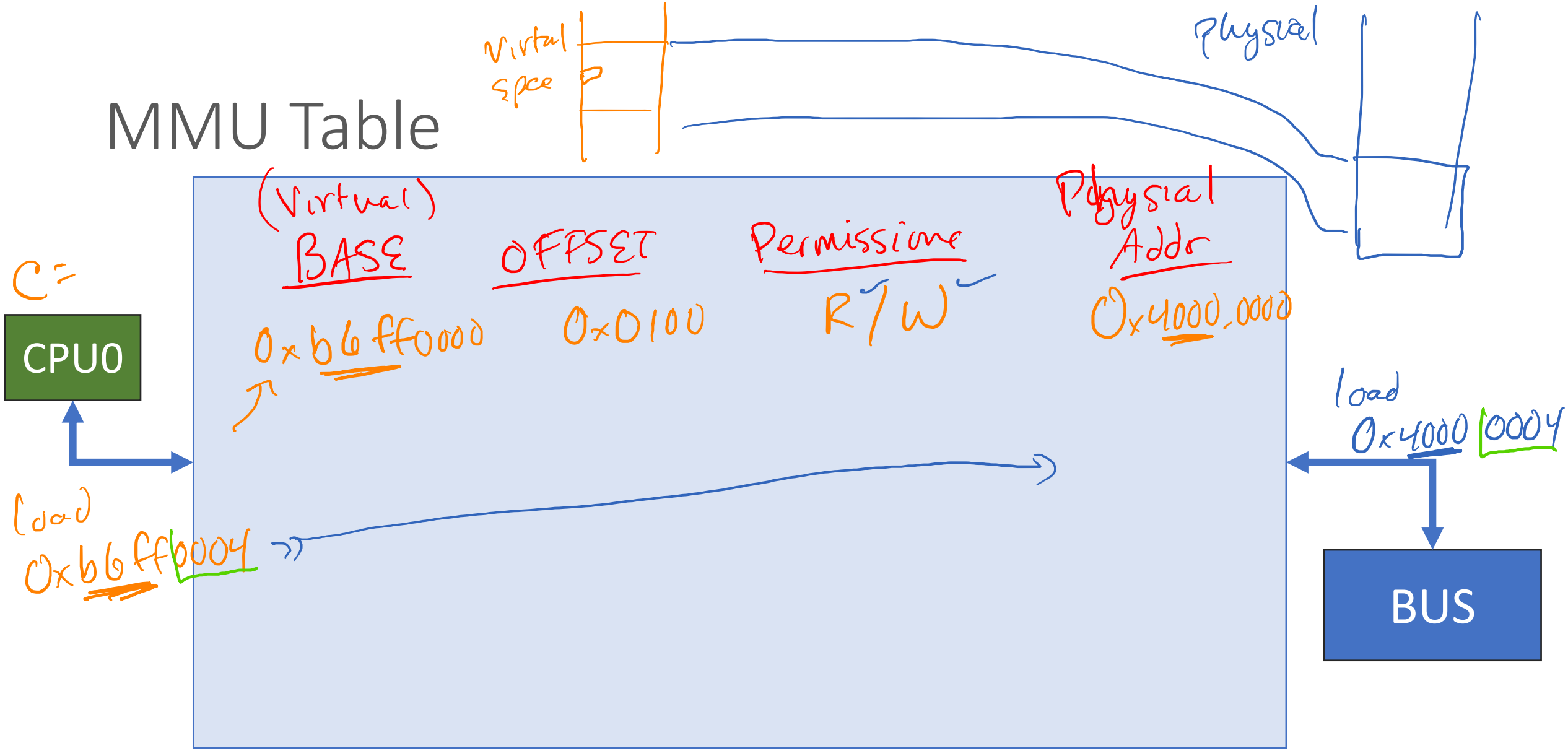
0x4000 0004
2000 0008

memory (mmio)

Physical Base	Offset

Physical Address

MMU Table


virtual space
Physical
(Virtual) BASE
OFFSET
Permissions
Physical Addr
0xb6ff0000
0x0100
R/W
0x4000.0000
C=
CPU0
load
0xb6ff0004
load
0x4000 0004
BUS

So how do we access the FPGA for P4?

- FPGA uses physical address
- CPU (w/Linux) uses virtual address

- Q: How do I talk to a physical address with Linux?

- A: Linux provides a special /dev/mem file to help us!

Next Time:

14: Memory Management

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University